

# Image Processing on AWS Lambda: Performance Evaluation and Cost Analysis

Nicolás Pacheco    Prajwal Shinde    Mathias Tirado  
ID: 2283582        ID: 2285327        ID: 2282472

*Cloud Computing, Prof. Emiliano Casalicchio  
Sapienza Università di Roma — June 2026*

## Abstract

We deployed three image-processing operations (*resize*, *grayscale conversion*, and *edge detection*) as independent AWS Lambda functions exposed through Amazon API Gateway, and evaluated their performance, scalability, and operational cost. A factorial load-testing matrix of 3 operations  $\times$  3 image sizes  $\times$  4 concurrency levels  $\times$  2 repetitions (72 scenarios) was executed with Locust from a local client machine reaching the Lambda endpoints over the public Internet, while five CloudWatch metrics per Lambda captured the server-side picture. Lambda processes every request in under 1.5 s with zero errors and zero throttles across 52,272 invocations, yet end users observe up to 46 s p95 response time and up to 57 % HTTP failures at 100 concurrent users in the worst-case scenario (resize-large). All failures were rejected upstream by API Gateway before reaching Lambda. A six-month cost projection places the break-even point against an always-on EC2 t3.small at roughly 5.5 M invocations per month.

## 1. Introduction

Serverless computing platforms such as AWS Lambda are increasingly adopted for event-driven, stateless workloads because they eliminate infrastructure management and bill only for actual execution time [1]. Image processing—characterised by variable payload sizes, purely stateless computation, and bursty request patterns—is a natural fit for this model, and serves as a concrete, measurable workload for evaluating serverless scalability.

This report describes the design, deployment, and empirical evaluation of an image-processing pipeline built on AWS Lambda following Option 2 of the course project. Three operations—resize, grayscale conversion, and edge detection—each run as a separate Lambda function exposed through Amazon API Gateway. The central evaluation question is *how a serverless deployment scales from one to one hundred concurrent users*, and crucially, how server-side Lambda metrics relate to the latency that end users actually perceive.

We executed 72 Locust load scenarios, cross-referenced client-side measurements with five per-Lambda CloudWatch metrics (Section 4–6), and projected the six-month operational cost against an EC2 baseline (Section 7). Our central finding: Lambda reported perfect server-side health while end users experienced up to 46 s response times and 27 % HTTP failures. This illustrates why monitoring a single layer of a multi-tier architecture produces a misleading picture of system health.

## 2. System Architecture

The system follows a **microservices** pattern: each operation runs in its own Lambda function with its own API Gateway endpoint. All three Lambdas share the same runtime—Python 3.11, x86\_64, 512 MB memory, 30-second function timeout—and use the public Klayers Pillow 11 layer for image manipulation [6]. API Gateway HTTP API endpoints retain the default 29-second integration timeout, which becomes the binding constraint at high concurrency (Section 6).

### 2.1. Per-operation algorithms

Each operation is a thin wrapper around Pillow primitives, intentionally keeping application logic minimal so that measurement focuses on infrastructure behaviour rather than algorithm tuning:

- **Resize** — LANCZOS resampling scales the input to a target width of 400 px while preserving aspect ratio. LANCZOS is the highest-quality interpolation filter in Pillow and is chosen to maximise per-invocation CPU work, making duration differences across image sizes more pronounced.
- **Grayscale** — `convert("L")` converts to single-channel 8-bit luminance, then re-encodes as JPEG at quality 85. The lightest of the three operations: each output pixel requires only a weighted sum of the three input colour channels.
- **Edge detection** — a  $3 \times 3$  SMOOTH averaging filter is first applied to reduce noise, followed by the FIND\_EDGES convolution kernel on the grayscale image [6]. Two sequential convolution passes make this the most CPU-intensive operation.

### 2.2. Microservices vs. monolithic Lambda

We initially prototyped a single monolithic Lambda in which the operation was selected by a request parameter, before migrating to the three-function decomposition. A microservice is characterised by *single responsibility*, *independent deployability*, *independent scalability* and *isolated failure modes*—properties that map naturally to AWS Lambda, where each function is by construction a separately deployed and separately scaled execution unit with its own IAM role, code package, and CloudWatch namespace. Our three-function decomposition therefore aligns Lambda’s runtime model with the microservices architectural style; a monolithic Lambda that routes between unrelated capabilities would borrow Lambda’s deployment model but lose those architectural properties.

The decisive consideration for this project, however, is *per-function observability*: CloudWatch publishes metrics in a per-function namespace, so a monolithic Lambda aggregates

Duration and Concurrent Executions across all three operations, making per-operation attribution impossible. With independent functions we can report that grayscale achieves  $p95 = 432$  ms while edge detection reaches  $p95 = 1,054$  ms (Table 1). This  $2.4\times$  gap would be unextractable from a shared namespace. The trade-off is a tripled deployment footprint (three functions, three endpoints, three IAM policies), which is acceptable given the measurement benefit.

### 3. Experimental Design

#### 3.1. Factorial matrix

The test matrix crosses three orthogonal factors: **operation** (resize, grayscale, edge)  $\times$  **image size** (small  $\approx 55$  KB, medium  $\approx 400$  KB, large  $\approx 1.3$  MB)  $\times$  **concurrent users** (1, 10, 50, 100), each combination repeated twice for variance estimation. Total:  $3 \times 3 \times 4 \times 2 = 72$  scenarios, each running for 2 minutes at steady state after a 10-second ramp-up. The original plan included 200 users and 5-minute runs; both were reduced to stay within the \$50 Vocareum credit budget. The 2-minute duration was sufficient for every scenario to reach a stable operating point, as confirmed by flat user-count curves in the per-scenario history files.

Test images were generated programmatically with Pillow using random-noise pixel fills. Solid-colour images were explicitly avoided because JPEG compression collapses them to a few kilobytes regardless of canvas dimensions, eliminating the payload-size variable. Random noise represents worst-case JPEG entropy and produces file sizes that reliably match the three target size categories.

#### 3.2. Data collection

Two complementary data sources were used in parallel throughout every scenario:

- **Locust [7] (client-side)** — per-scenario `stats.csv` with aggregated  $p50/p95/p99$  response times, throughput, and failure counts, plus a per-second `stats_history.csv` timeline. Captures the full end-to-end latency including network transit and all queuing delays between the load generator and Lambda.
- **CloudWatch [4] (server-side)** — per Lambda, five metrics at 1-minute granularity: Invocations (Sum), Duration (Min/Avg/Max), Concurrent Executions (Max), Errors (Sum), and Throttles (Sum). Captures only what happens inside the Lambda execution environment, with no visibility into upstream queuing.

All timestamps were normalised to Europe/Rome (UTC+2, June 2026) before cross-referencing the two datasets.

#### 3.3. Load generator placement

Locust ran on a team member’s personal laptop in Italy, connecting to the three Lambda functions in `us-east-1` through their API Gateway HTTPS endpoints over the public Internet. The reported response times therefore include real-world network round-trip latency in addition to queueing and Lambda execution time, which we consider representative of how a typical web user would experience the service.

### 4. Server-side Results (CloudWatch)

Figure 1 and Table 1 present the per-Lambda CloudWatch metrics aggregated over the full 3-hour test session.

Three observations stand out. First, Duration is **remarkably stable** across the entire session:  $p95$  sits at 432–

**Table 1:** Per-Lambda server-side summary (CloudWatch, full session).

| Metric               | resize | gray   | edge    |
|----------------------|--------|--------|---------|
| Total invocations    | 17,342 | 17,521 | 17,409  |
| Avg Duration (ms)    | 271.1  | 142.8  | 438.0   |
| p95 Duration (ms)    | 615.8  | 432.3  | 1,053.8 |
| Min Duration (ms)    | 11.2   | 1.6    | 8.5     |
| Max Duration (ms)    | 856.4  | 740.3  | 1,431.3 |
| Max concurrent exec. | 11     | 10     | 9       |
| Total errors         | 0      | 0      | 0       |
| Min success rate (%) | 100    | 100    | 100     |
| Total throttles      | 0      | 0      | 0       |

1,054 ms depending on the operation and never spikes in response to increasing load, as visible in Figure 1b. This confirms that Lambda’s execution environment scales horizontally without degradation—each additional concurrent invocation receives its own container and executes at the same speed as a lone request. Second, **peak concurrent executions reached only 9–11** (Figure 1a), two orders of magnitude below the 1,000-concurrent production default [2]. The Vocareum sandbox imposes an account-level concurrency cap that manifests as this ceiling; its practical effect is that requests accumulate in the API Gateway queue rather than spawning additional Lambda instances. This is the root cause of the client-side failures discussed in Section 5. Third, **Lambda reported zero errors, zero throttles, and 100 % success** across all 52,272 invocations. From the function’s perspective, the service operated perfectly throughout.

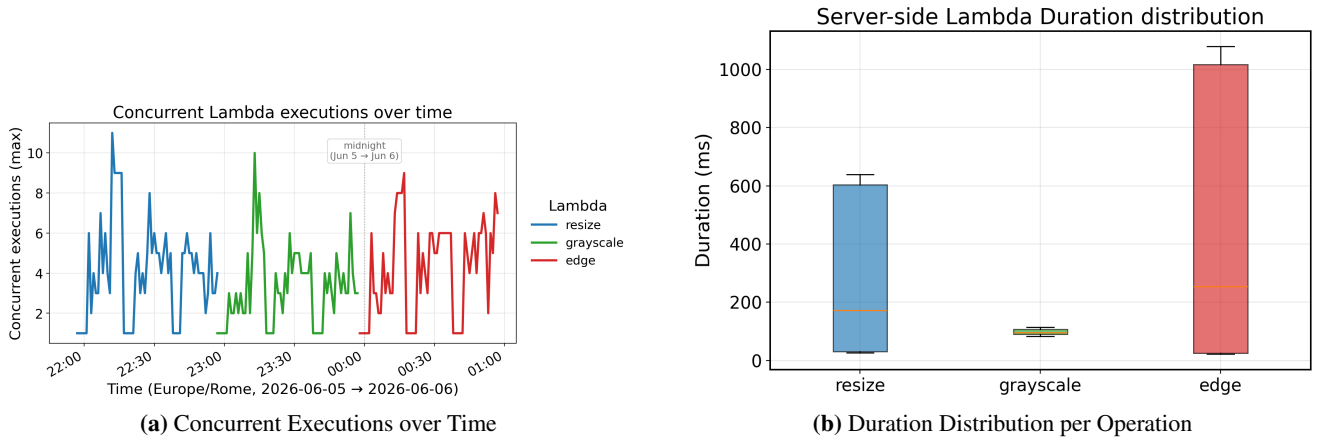
The Duration hierarchy is consistent with algorithmic complexity: grayscale (143 ms average) performs a single per-pixel luma conversion; resize (271 ms) applies a multi-tap LANCZOS resampling kernel; edge detection (438 ms) runs two sequential convolution passes. Because Lambda bills per millisecond of execution time at the allocated memory, this ordering maps directly to a proportional cost differential, quantified in Section 7.

### 5. Client-side Results (Locust)

Figure 2 presents the three primary client-side metrics. The contrast with the server-side picture is stark. While Lambda was completing each individual invocation in under 1.5s (Table 1), end-to-end  $p95$  response time grew non-linearly with the user count, reaching  $\approx 46$ s at 100 concurrent users for all three operations (Figure 2a). The three operation curves are nearly coincident at high concurrency, indicating that operation type becomes irrelevant once the API Gateway queue is saturated: a 143 ms grayscale call and a 438 ms edge call both wait the same 45+ seconds in queue before they are even dispatched.

Throughput tells the same story from a supply-side perspective (Figure 2b): completed requests per second plateau well below the nominal user count because each Locust virtual user is serialised on its own response. Adding virtual users beyond the saturation point increases queue depth without increasing throughput, a classic symptom of a queueing-limited system.

Across the 72 scenarios Locust recorded **315 failures** (Figure 2c): 161 HTTP 0 (TCP connection reset before any response) and 154 HTTP 408 (request timeout returned by



**Figure 1:** Server-side metrics from AWS CloudWatch. (a) Concurrent Lambda executions over the full 3-hour session; peak concurrency of 9–11 is visible at the busiest scenarios and sits two orders of magnitude below the 1,000-concurrent production default, confirming that the Vocareum sandbox concurrency cap is the binding scalability constraint. (b) Duration distribution per operation across all 72 scenarios; grayscale is the lightest ( $\bar{x} = 143$  ms), edge detection the heaviest ( $\bar{x} = 438$  ms), and the spread within each operation remains narrow, confirming that Lambda execution time is stable regardless of load level.

API Gateway after 29 s). The worst single scenario was *resize\_large at 100 users*, with 82 failures in the first repetition and 61 in the second (a 55 % failure rate in the second run, 61 out of 111 requests). Failures were entirely absent at 1 and 10 users and first appeared at 50 users, consistent with the hypothesis that they are caused by API Gateway queue exhaustion rather than any instability within Lambda.

The effect of payload size on client-perceived latency is summarised in Table 2. At moderate load (10 users) the size effect is mild and proportional to the payload; at peak load (100 users), larger images inflate p95 response time by more than an order of magnitude across all three operations, because the per-request service time amplifies into seconds of additional queue wait for every other request behind it.

## 6. Cross-reference: Where Does the Latency Live?

Figure 3a overlays the two p95 series—Locust end-to-end (solid lines) and CloudWatch Lambda Duration (dashed lines)—for each operation across all concurrency levels. The gap between solid and dashed represents everything that happens outside pure Lambda execution: API Gateway request routing, TCP connection establishment, payload serialisation, network round-trip, and most importantly *queue wait time*

**Table 2:** Client-side p95 response time (ms) by image size, at moderate (10 users) and peak (100 users) load. S/M/L = small/medium/large images. Values averaged across repetitions.

| Op.       | 10 users |       |        | 100 users |        |        |
|-----------|----------|-------|--------|-----------|--------|--------|
|           | S        | M     | L      | S         | M      | L      |
| resize    | 320      | 2,000 | 12,000 | 3,050     | 41,000 | 96,500 |
| grayscale | 285      | 2,000 | 13,500 | 4,100     | 35,000 | 99,000 |
| edge      | 315      | 2,100 | 12,500 | 3,150     | 38,000 | 97,500 |

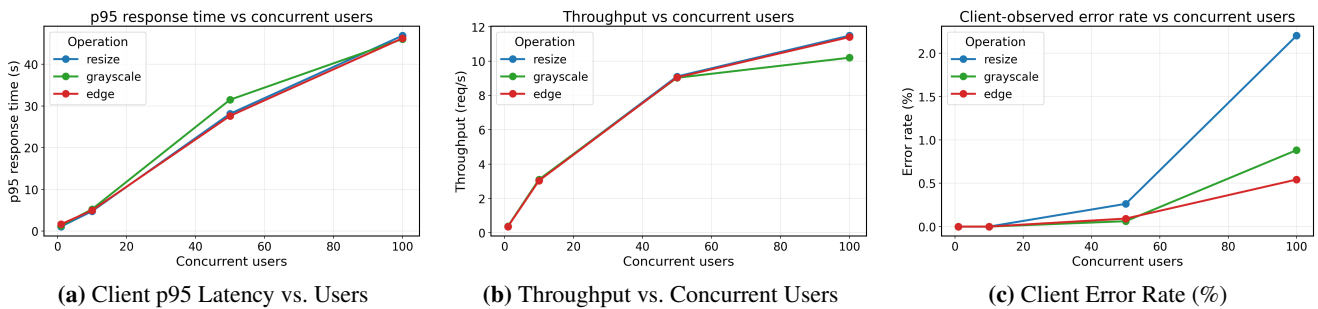
once the concurrency cap is reached.

For each scenario we formalise this decomposition as:

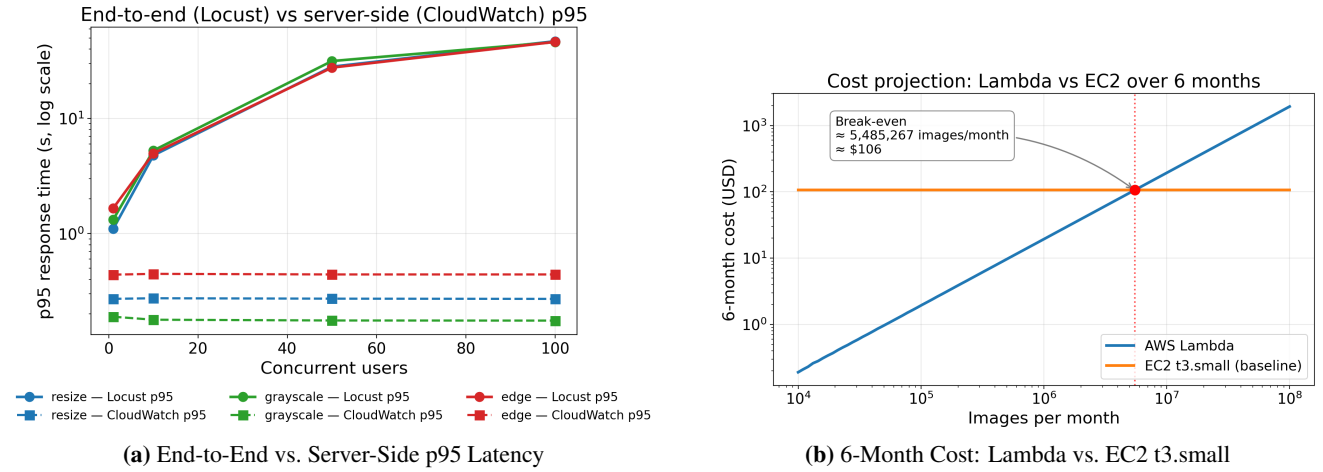
$$\text{overhead} = \text{Locust p95} - \text{CloudWatch p95}$$

which approximates the sum of all non-Lambda contributions to end-to-end latency. Table 3 reports the result aggregated by operation and user level, averaged across image sizes and repetitions.

Even at a single concurrent user, 74–86 % of end-to-end latency is overhead unrelated to Lambda execution, reflecting fixed costs of TCP handshake, TLS negotiation, and API Gateway routing. By 10 users the overhead fraction rises



**Figure 2:** Client-side performance captured by Locust across all 72 scenarios. (a) p95 response time grows non-linearly from  $\approx 1$  s at 1 user to  $\approx 46$  s at 100 users for all three operations, despite Lambda execution remaining stable (cf. Table 1). The three operation curves converge at high concurrency, confirming that the bottleneck is the upstream queue, not the function logic. (b) Throughput saturates well below the nominal user count because each virtual user is blocked waiting for its previous response. (c) HTTP failures are absent at 1 and 10 users, appear at 50, and reach up to 57 % for *resize\_large* at 100 users (55 % in the second repetition, 61 out of 111 requests); all originate in API Gateway, not in Lambda.



**Figure 3:** (a) End-to-end p95 (solid lines, Locust) vs. server-side p95 (dashed lines, CloudWatch) per operation and concurrency level. The gap between solid and dashed is the API Gateway queuing overhead; at 100 users it accounts for >99 % of observed latency for all three operations. Note that the dashed lines for all three operations are nearly flat and indistinguishable near the x-axis, confirming that Lambda Duration does not increase with load. (b) Six-month cost projection. Lambda cost grows linearly with invocation volume starting near zero; the always-on EC2 t3.small incurs a flat  $\approx \$106$  regardless of traffic. Break-even occurs at  $\approx 5.5$  M invocations per month, above which provisioned capacity becomes the rational economic choice.

**Table 3:** End-to-end latency decomposition (ms). Overhead = Locust p95 – CloudWatch p95. %OH = overhead as a percentage of total end-to-end latency. Values averaged across the three image sizes and two repetitions per cell.

| Operation | Users | Locust p95 (ms) | CW p95 (ms) | Overhead (ms) | %OH    |
|-----------|-------|-----------------|-------------|---------------|--------|
| resize    | 1     | 1,097           | 272         | 826           | 75 %   |
| resize    | 10    | 4,773           | 274         | 4,499         | 94 %   |
| resize    | 50    | 28,133          | 272         | 27,861        | 99 %   |
| resize    | 100   | 46,850          | 270         | 46,580        | 99.4 % |
| grayscale | 1     | 1,323           | 189         | 1,134         | 86 %   |
| grayscale | 10    | 5,262           | 178         | 5,083         | 97 %   |
| grayscale | 50    | 31,492          | 176         | 31,316        | 99 %   |
| grayscale | 100   | 46,033          | 175         | 45,858        | 99.6 % |
| edge      | 1     | 1,660           | 439         | 1,221         | 74 %   |
| edge      | 10    | 4,972           | 446         | 4,525         | 91 %   |
| edge      | 50    | 27,618          | 441         | 27,177        | 98 %   |
| edge      | 100   | 46,216          | 441         | 45,775        | 99.0 % |

to 91–97 %; by 50–100 users it exceeds 99 % for every operation. At this point Lambda Duration—though perfectly stable—is effectively invisible to the end user. One practical implication: optimising the Lambda function code (e.g., switching LANCZOS to a faster interpolation filter) would produce *no measurable improvement* in user-facing latency under any non-trivial load, because the bottleneck is the upstream queue, not the function itself. The correct remedy is to raise the account-level concurrency limit or adopt AWS Lambda Provisioned Concurrency to pre-warm containers.

This behaviour is consistent with Little’s Law applied to the upstream queue. With an arrival rate  $\lambda$  that grows with the number of concurrent users and a service capacity bounded by the observed concurrency ceiling  $C \approx 10$ , the queue settles to a steady-state length of roughly  $\lambda \cdot \bar{W}$ , where  $\bar{W}$  is the per-request service time. Once  $\lambda$  exceeds  $C/\bar{W}$  the queue saturates and the wait time  $W = N/\lambda$  grows linearly with the backlog  $N$ , eventually triggering the API Gateway 29 s integration timeout. This is precisely the regime in which the HTTP 408 failures of Section 5 appear.

## 6.1. Two perspectives on success

The coexistence of a perfect CloudWatch record with 315 client-side failures is not a contradiction. It reflects that the two systems count fundamentally different events. CloudWatch counts Lambda invocations that started and completed within the function timeout. Locust counts every HTTP response that was not a 2xx, regardless of which layer produced it. All 315 failures occurred upstream of Lambda: 161 HTTP 0 responses indicate API Gateway closed the TCP connection without forwarding the request to any Lambda (queue exhausted, connection reset); 154 HTTP 408 responses indicate the 29-second integration timeout [3] fired while the request was still waiting in queue. The longest individual Lambda execution was 1,431 ms for edge detection. The 30-second function timeout was never approached across any of the 52,272 invocations. A deployment that exposes only Lambda metrics would therefore present a falsely healthy picture of a system that, at peak load, was rejecting more than one in two user requests at the gateway in the worst-case scenario.

## 7. Cost Projection — Lambda vs. EC2

Figure 3b visualises the six-month cost comparison between the serverless deployment and an equivalent always-on EC2 baseline. Using the measured average Durations (resize 271 ms, grayscale 143 ms, edge 438 ms; mean 284 ms across operations) we apply public on-demand pricing [1, 5]:

- **Lambda:** \$0.20 per million invocations plus  $\$1.67 \times 10^{-5}$  per GB-second at 512 MB allocation. The first 1 M invocations per month and 400,000 GB-seconds are covered by the AWS Free Tier indefinitely.
- **EC2 t3.small:** \$0.0208 / hour on-demand in `us-east-1`  $\approx$  \$17.67/month, regardless of request volume, running 24/7. Over six months this totals  $\approx$  \$106.

Table 4 materialises the calculation at representative monthly volumes, applying the formula above to the mean measured Duration of 284 ms and 512 MB memory.

**Table 4:** Six-month deployment cost at representative monthly volumes. Lambda cost computed as  $\$0.20 \cdot N/10^6 + \$1.67 \times 10^{-5} \cdot N \cdot D \cdot M/1024$ , where  $N$  = invocations,  $D$  = avg Duration (s),  $M$  = memory (MB), multiplied by 6 months.

| Inv./month | Lambda (6m) | EC2 (6m) | Winner     |
|------------|-------------|----------|------------|
| 100 K      | \$1.2       | \$106    | Lambda     |
| 1 M        | \$19        | \$106    | Lambda     |
| 5 M        | \$95        | \$106    | Lambda     |
| 5.5 M      | \$106       | \$106    | break-even |
| 10 M       | \$190       | \$106    | EC2        |
| 100 M      | \$1,938     | \$106    | EC2        |

Lambda cost starts near zero and grows linearly with traffic; EC2 cost is a flat step function. The **break-even** falls at roughly **5.5 M invocations per month**: below that threshold Lambda is economically rational; above it, a provisioned instance becomes cheaper, provided a single t3.small can sustain the required response-time SLA at the corresponding request rate, a condition the present study has not verified. For mixed-operation workloads the break-even shifts depending on the fraction of edge-detection calls, which cost 3× more in Lambda Duration than grayscale.

## 8. Discussion and Limitations

**Vocareum sandbox concurrency cap.** The account-level concurrency ceiling clamped peak concurrent executions to 9–11, two orders of magnitude below the 1,000-concurrent production default [2]. This is the direct cause of the API Gateway queue saturation observed from 50 users onward. In a production account with the default limit, the same 100-user load would spawn up to 100 parallel Lambda containers, each completing in under 1.5 s, with no queue accumulation and no client-visible failures. The sandbox thus stress-tests the upstream queuing layer rather than Lambda’s own scalability, and the latency and failure numbers cannot be directly extrapolated to a real-world deployment.

**Load generator over the public Internet.** Locust ran on a personal laptop in Italy reaching the Lambda endpoints in `us-east-1` over the public Internet, so the reported response times already include the round-trip network latency that a real user would also experience. Our overhead values (Table 3) therefore embed a fixed RTT component that is independent of Lambda or queuing. Users with different ge-

ographic distance to `us-east-1` would see proportionally different base latency.

**Operation cost proportional to CPU complexity.** Edge detection ran 62 % longer than resize and 207 % longer than grayscale on average. Because Lambda bills per millisecond at the allocated memory, this translates directly into proportional cost differentials. A production deployment processing a mixed workload should weight the cost projection by the expected operation mix rather than using a single average Duration.

**Missing cold-start measurement.** A dedicated cold-start probe was excluded due to budget constraints. Such a probe would invoke each Lambda after a  $\geq 20$ -minute idle period to force cold container initialisation, then record Init Duration from CloudWatch Logs Insights. Init Duration is explicitly cited in the course brief and its absence is a recognised gap. Future work should reserve  $\approx 5$  % of the credit budget for 10 cold-start observations per Lambda, sufficient to characterise initialisation overhead and its contribution to worst-case p99 latency.

## 9. Conclusions

This study deployed and evaluated three image-processing Lambda functions under a controlled factorial load-test. The four principal findings are:

1. **Server-side health does not imply user-side health.** Lambda reported zero errors and 100 % success across 52,272 invocations, yet the same workload produced 46 s p95 latency and up to 57 % HTTP failures for end users in the worst-case scenario at 100 concurrent users. All failures originated in API Gateway and are invisible to Lambda-only monitoring.
2. **Queueing overhead dominates at scale.** At 50–100 concurrent users, API Gateway queueing accounts for  $>99$  % of observed end-to-end latency (Table 3, Figure 3a). Code-level Lambda optimisations have negligible impact; the correct remedy is raising the concurrency limit or adopting Provisioned Concurrency.
3. **Operation type determines cost, not user-facing latency at scale.** Grayscale, resize, and edge differ by up to 3× in Lambda Duration—and therefore billing cost—but their client-side p95 values converge to  $\approx 46$  s at high concurrency because the bottleneck shifts entirely to the upstream queue (Figure 2a).
4. **Lambda is cost-effective below  $\approx 5.5$  M invocations/month.** Above that threshold an always-on EC2 t3.small is cheaper, subject to latency SLA verification at the corresponding load (Figure 3b).

The methodological lesson is that evaluating a serverless deployment requires *both* a server-side and a client-side perspective. A Locust load generator combined with CloudWatch metric export, cross-referenced as shown in Section 6, is the minimum instrumentation needed to arrive at a correct and complete characterisation of system behaviour.

## References

- [1] Amazon Web Services. *AWS Lambda Pricing*. <https://aws.amazon.com/lambda/pricing/> Accessed June 2026.
- [2] Amazon Web Services. *Managing AWS Lambda Function Concurrency*. AWS Lambda Developer Guide.

<https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html>

- [3] Amazon Web Services. *Amazon API Gateway Quotas and Important Notes*. <https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html>
- [4] Amazon Web Services. *Using Amazon CloudWatch Metrics with AWS Lambda*. <https://docs.aws.amazon.com/lambda/latest/dg/monitoring-metrics.html>
- [5] Amazon Web Services. *Amazon EC2 On-Demand Pricing — t3.small, us-east-1*. <https://aws.amazon.com/ec2/pricing/on-demand/>
- [6] Pillow Contributors. *Pillow (PIL Fork) Documentation — Image Filters and Resize*. <https://pillow.readthedocs.io/> Accessed June 2026.
- [7] Locust Contributors. *Locust — A Modern Load Testing Framework*. <https://locust.io/> Accessed June 2026.